

La classe Thread

Fabrice.Kordon@lip6.fr



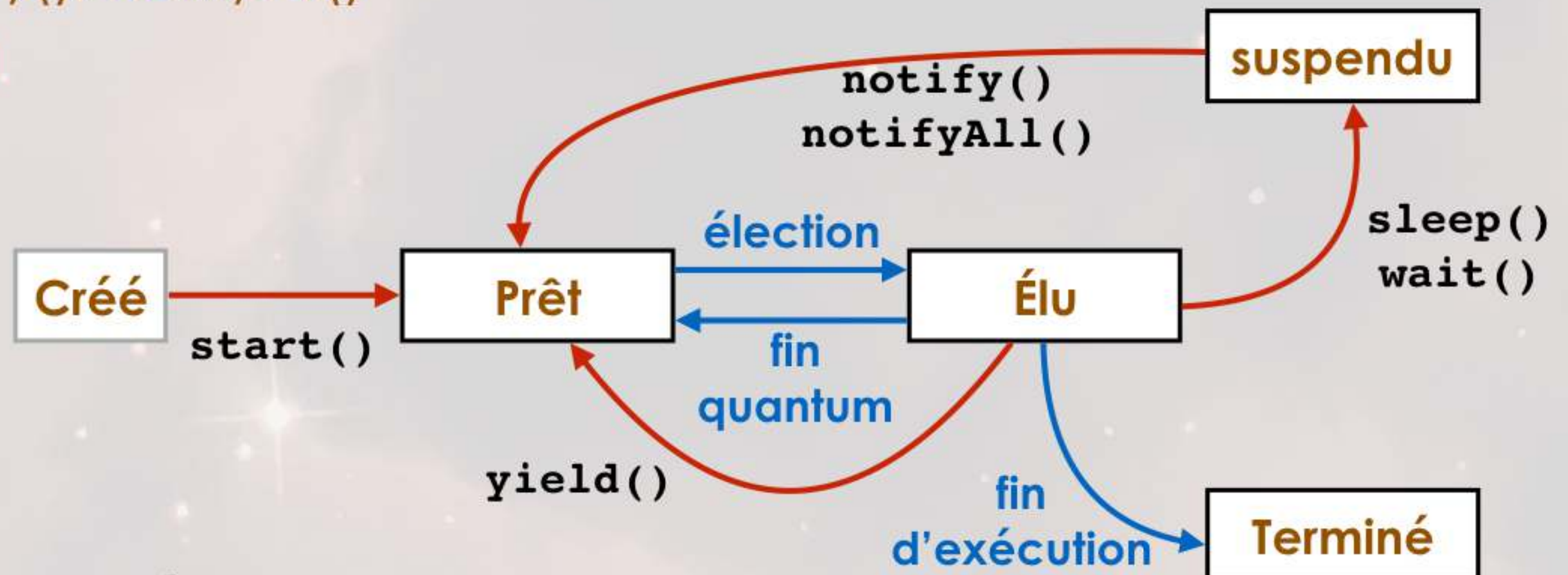
Présentation de la classe Thread

2

Objet de base permettant le parallélisme

-  `start()`
-  `sleep()`
-  `yield()`
-  `wait()`, `notify()`, `notifyAll()`

► vu plus tard



Corps de la tâche

-  Méthode `run()`

► Appelée par `start()`

Un exemple simple

3

```
class uneTache extends Thread {
    String monNom;

    uneTache (String n) {
        monNom = n;
    }

    public void run() {
        System.out.println ("Bonjour de " + monNom);
        System.out.println ("Au revoir de " + monNom);
    }
}

public class ExempleThread0 {
    public static void main(String []args) {
        uneTache t1, t2, t3;

        t1 = new uneTache("t1");
        t2 = new uneTache("t2");
        t3 = new uneTache("t3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Un exemple simple

3

```
class uneTache extends Thread {
    String monNom;

    uneTache (String n) {
        monNom = n;
    }

    public void run() {
        System.out.println ("Bonjour de " + monNom);
        System.out.println ("Au revoir de " + monNom);
    }
}

public class ExempleThread0 {
    public static void main (String[] args) {
        new uneTache("t1").start();
        new uneTache("t2").start();
        new uneTache("t3").start();
    }
}
```

Exécutions différentes?

C'est normal!

```
$ java ExempleThread0
Bonjour de t1
Au revoir de t1
Bonjour de t3
Bonjour de t2
Au revoir de t2
Au revoir de t3
$ java ExempleThread0
Bonjour de t1
Au revoir de t1
Bonjour de t3
Au revoir de t3
Bonjour de t2
Au revoir de t2
$ java ExempleThread0
Bonjour de t2
Au revoir de t2
Bonjour de t1
Au revoir de t1
Bonjour de t3
Au revoir de t3
```

Chronogramme de l'exécution

4

main



```
public class ExempleThread0 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```



```
$ java ExempleThread1
```

Chronogramme de l'exécution

4



```
public class ExempleThread0 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        ▶ t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains the command: `$ java ExempleThread1`

Chronogramme de l'exécution

4



```
public class ExempleThread0 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains the command: `$ java ExempleThread1`

Chronogramme de l'exécution

4



```
public class ExempleThread0 {
    public static void main(String []args) {
        uneTache t1, t2, t3;

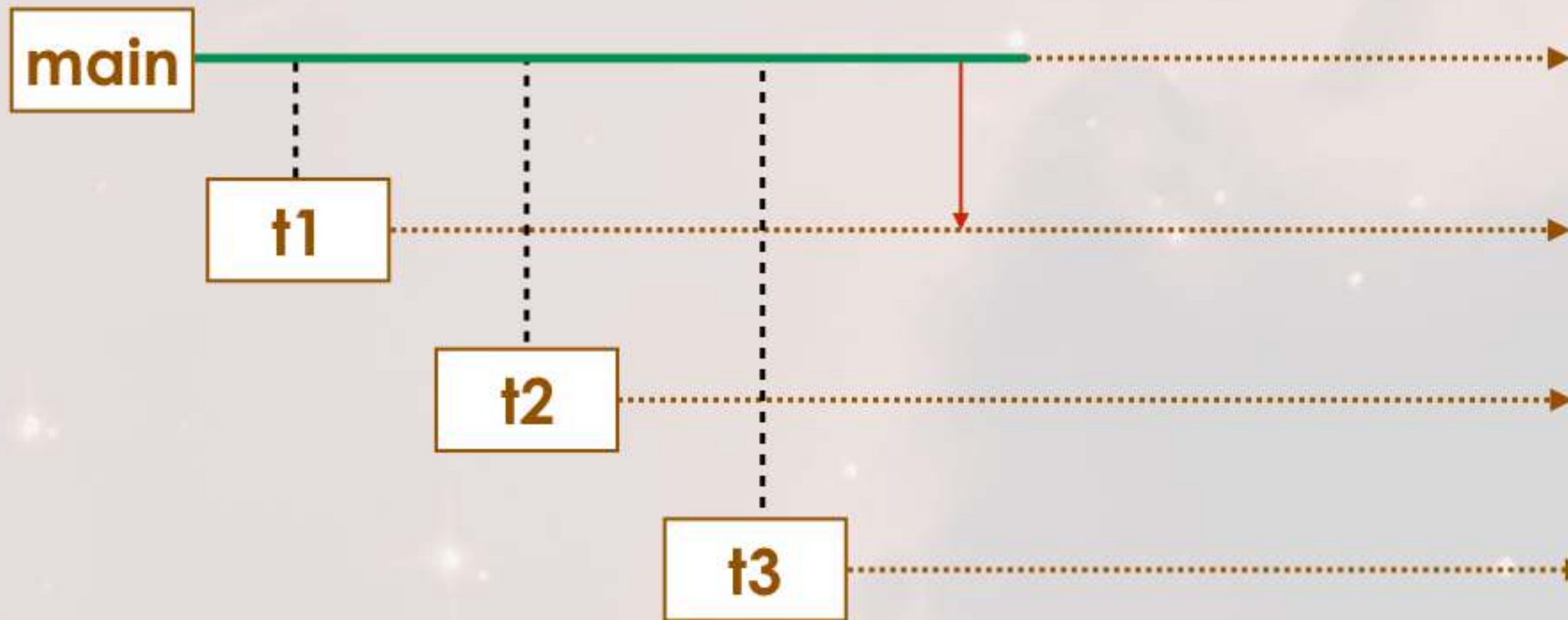
        t1 = new uneTache("t1");
        t2 = new uneTache("t2");
        t3 = new uneTache("t3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

```
$ java ExempleThread1
```

Chronogramme de l'exécution

4

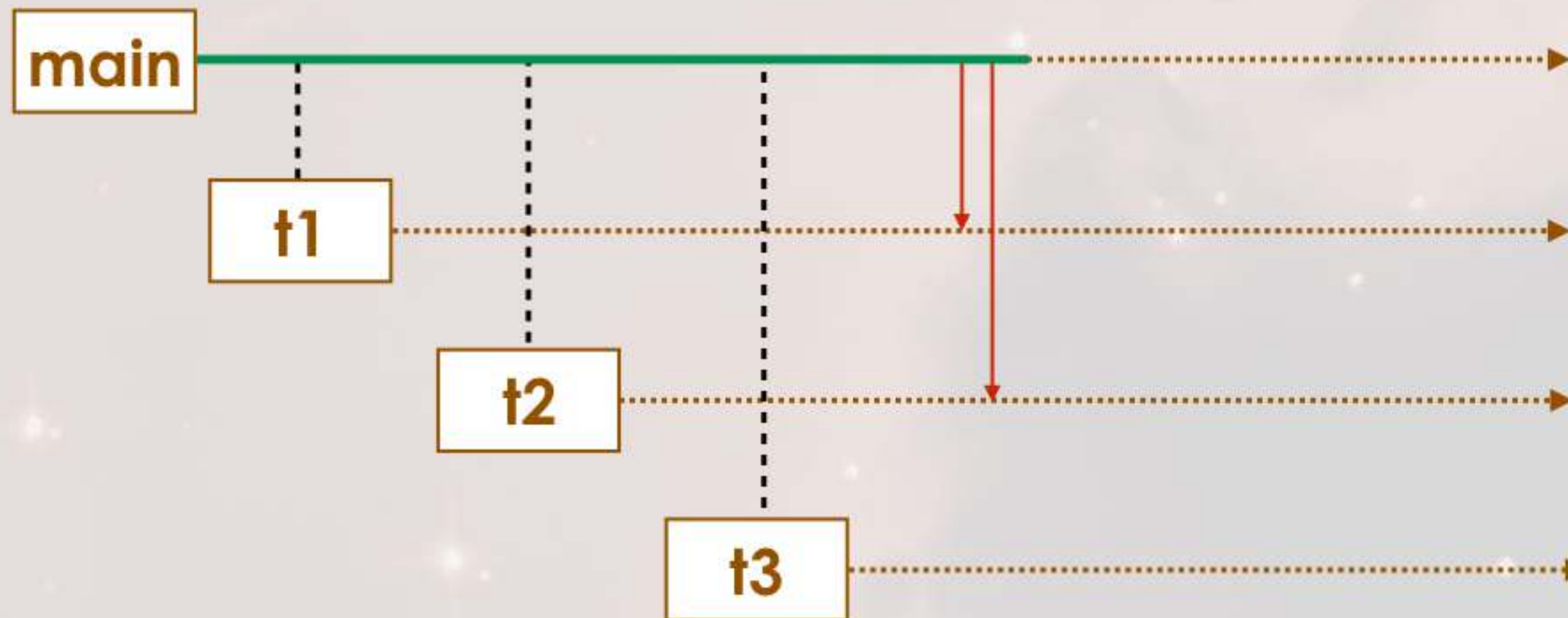


```
public class ExempleThread0 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleThread1
```

Chronogramme de l'exécution

4



```
public class ExempleThread0 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleThread1
```

Chronogramme de l'exécution

4

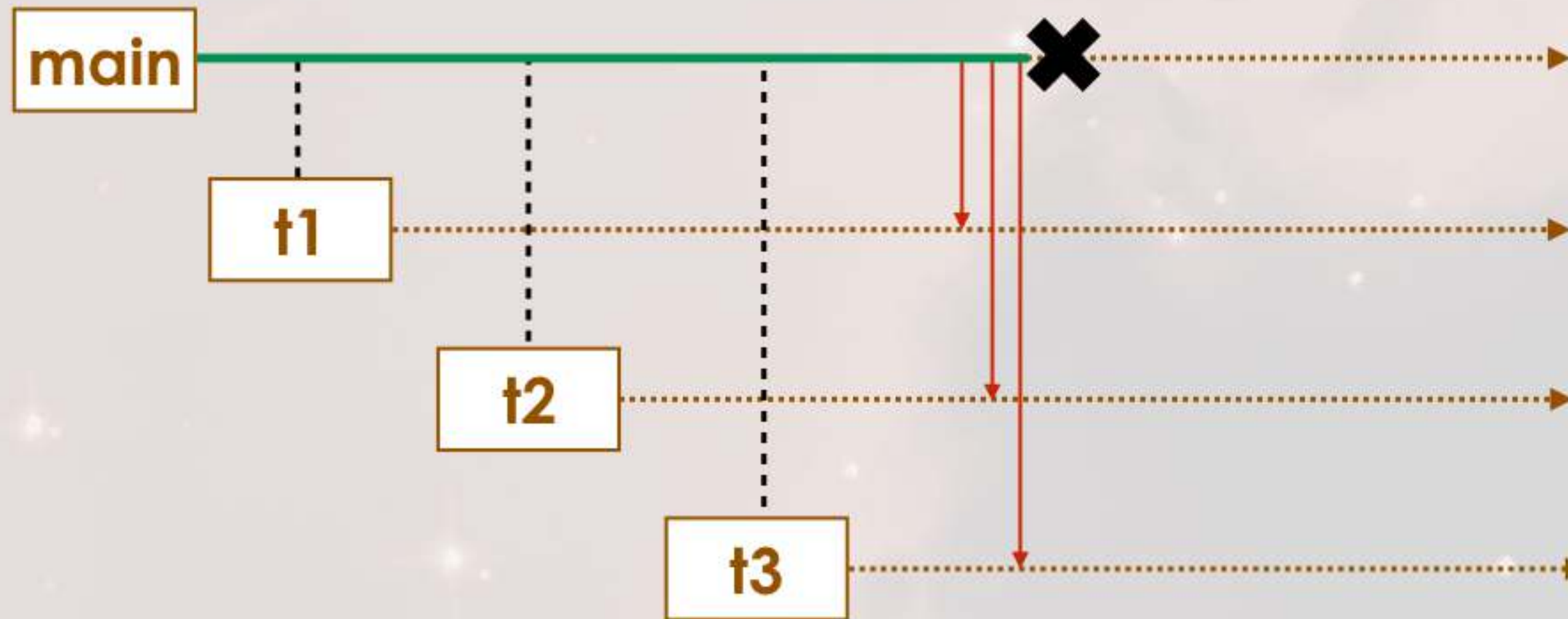


```
public class ExempleThread0 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It contains the command to run the Java program: `$ java ExempleThread1`.

Chronogramme de l'exécution

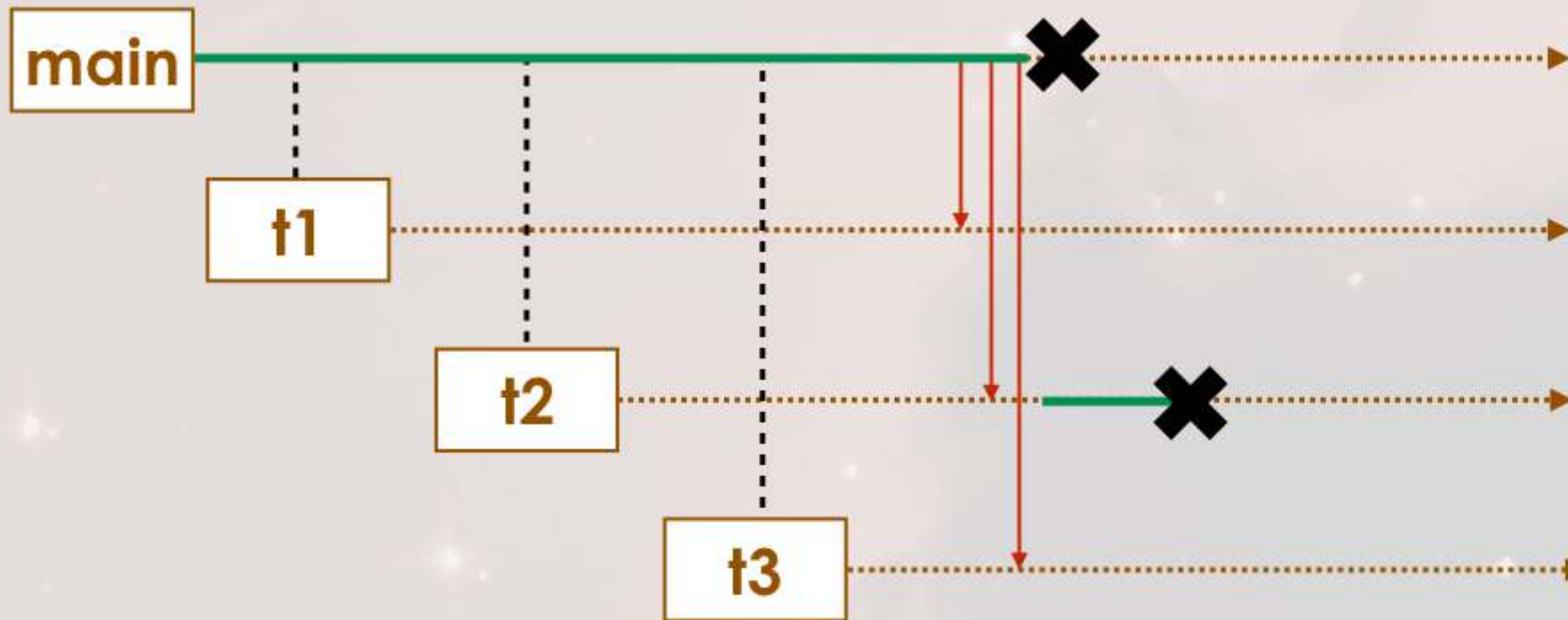
4



```
$ java ExempleThread1
```

Chronogramme de l'exécution

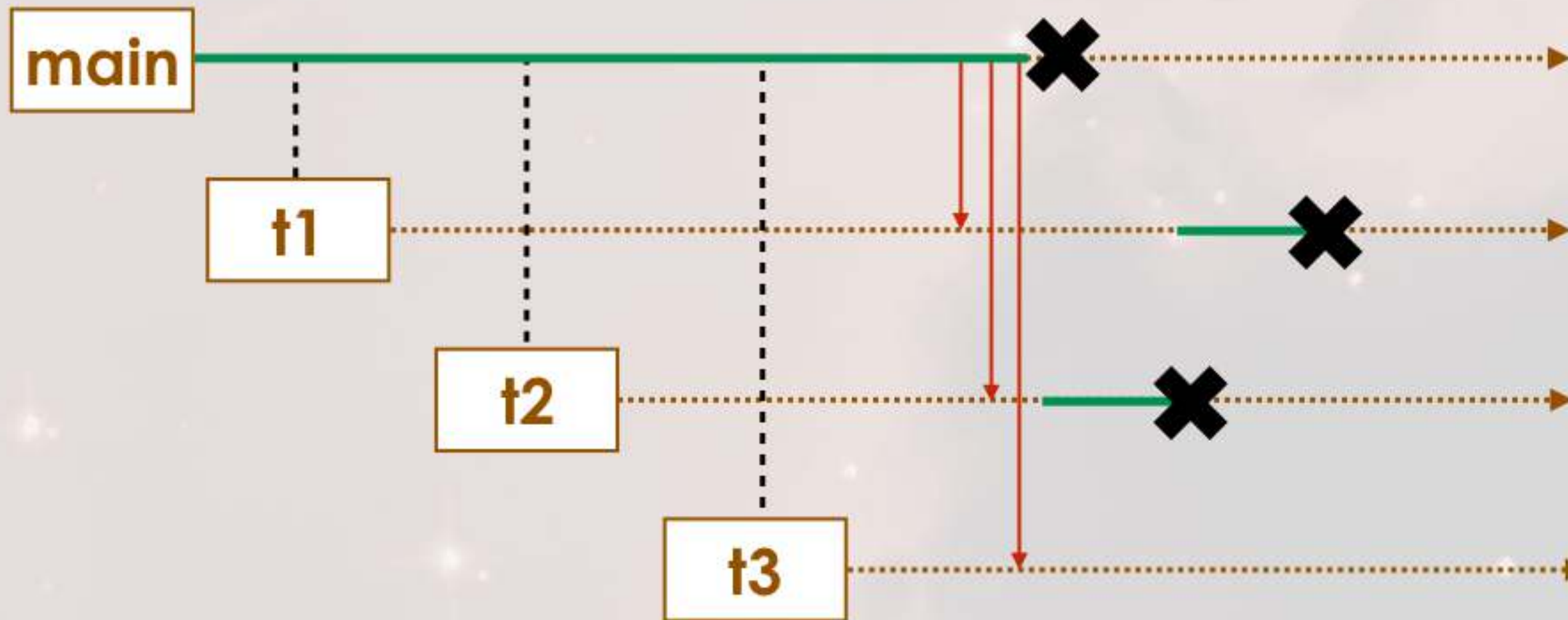
4



```
$ java ExempleThread1  
Bonjour de t2  
Au revoir de t2
```

Chronogramme de l'exécution

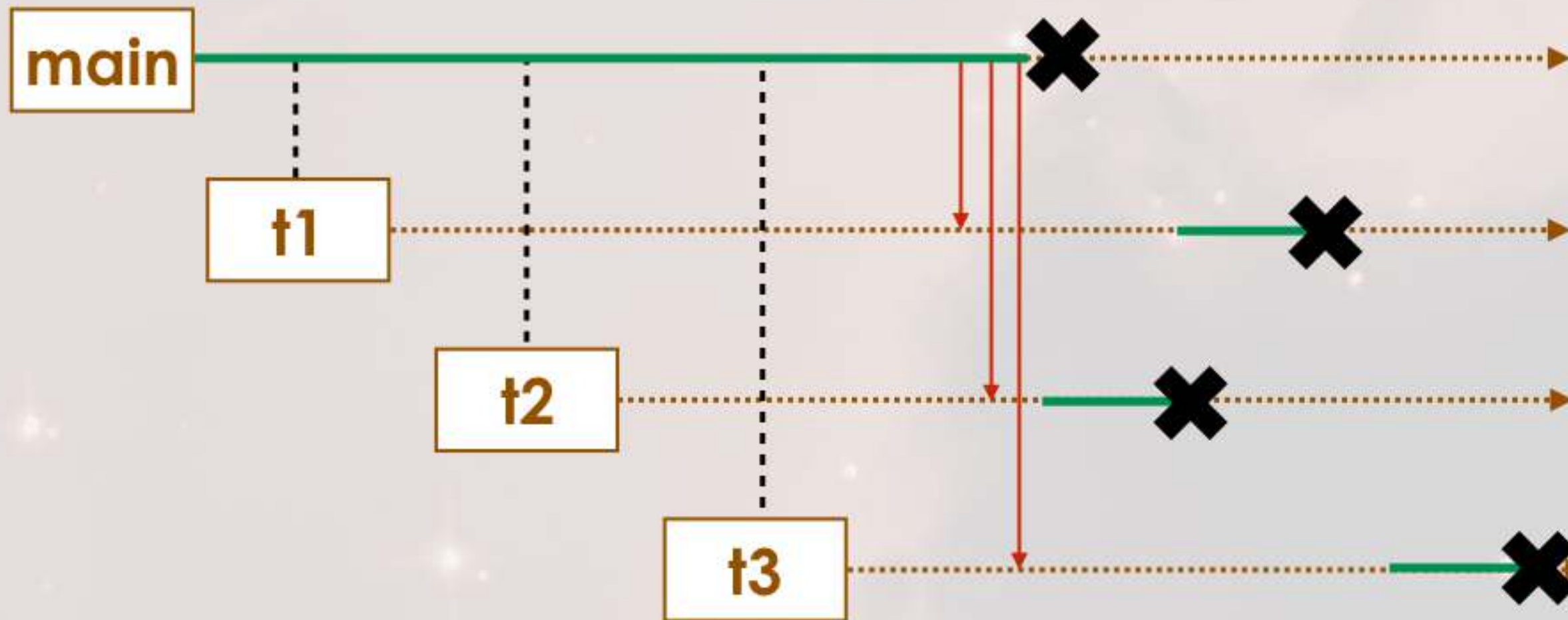
4



```
$ java ExempleThread1  
Bonjour de t2  
Au revoir de t2  
Bonjour de t1  
Au revoir de t1
```

Chronogramme de l'exécution

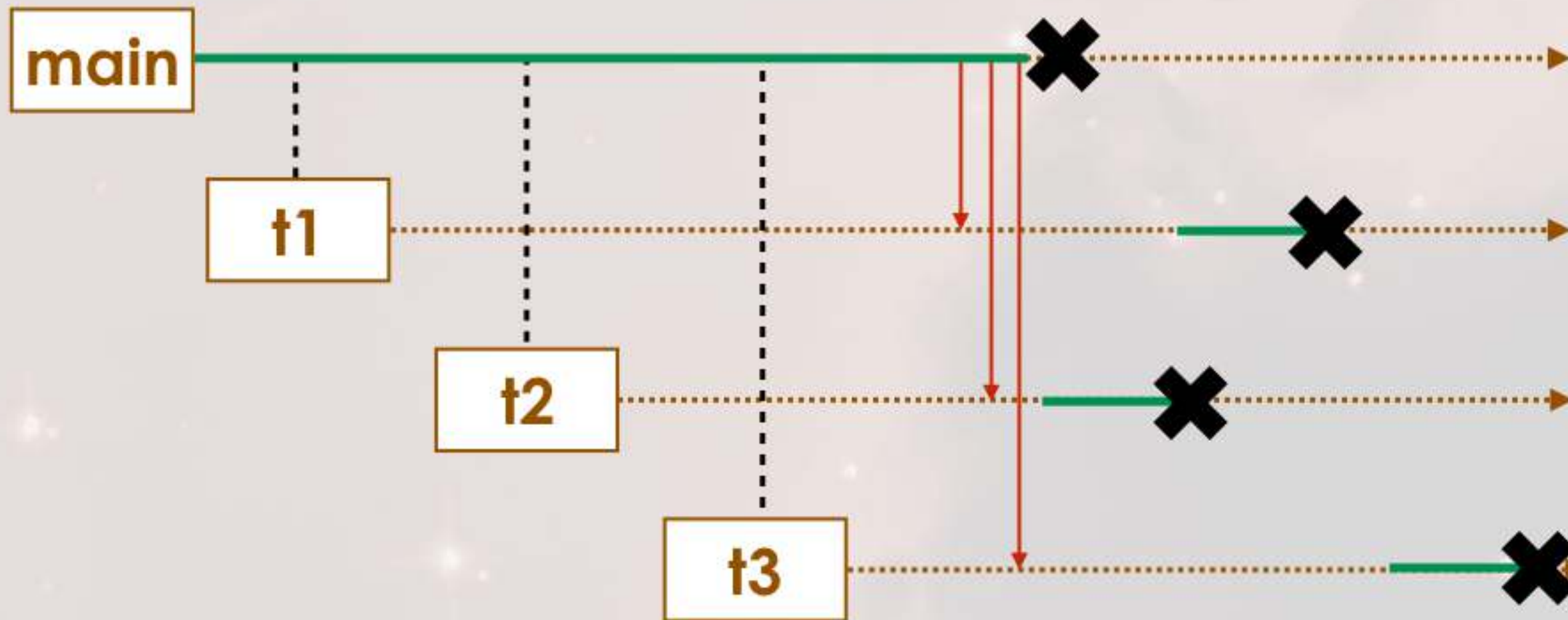
4



```
$ java ExempleThread1  
Bonjour de t2  
Au revoir de t2  
Bonjour de t1  
Au revoir de t1  
Bonjour de t3  
Au revoir de t3
```

Chronogramme de l'exécution

4



```
$ java ExempleThread1  
Bonjour de t2  
Au revoir de t2  
Bonjour de t1  
Au revoir de t1  
Bonjour de t3  
Au revoir de t3
```

C'est fini!

Le programme se termine
Le processus est nettoyé par l'OS

Variation avec commutation forcée

5

```
class uneTache extends Thread {
    String monNom;

    uneTache (String n) {
        monNom = n;
    }

    public void run() {
        System.out.println ("Bonjour de " + monNom);
        Thread.yield();
        System.out.println ("Au revoir de " + monNom);
    }
}

public class ExempleThread1 {
    public static void main(String []args) {
        uneTache t1, t2, t3;

        t1 = new uneTache("t1");
        t2 = new uneTache("t2");
        t3 = new uneTache("t3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Variation avec commutation forcée

5

```
class uneTache extends Thread {
    String monNom;

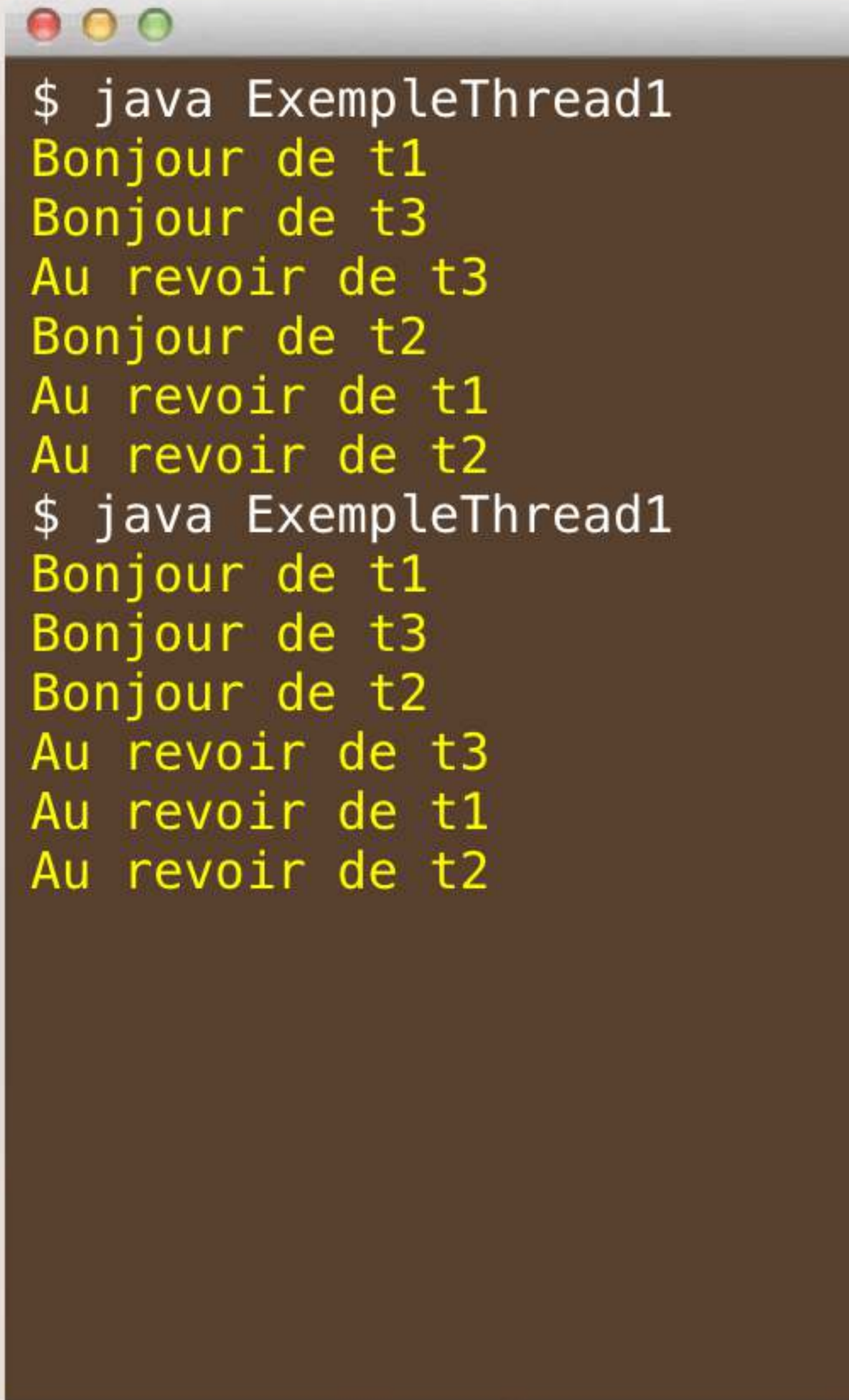
    uneTache (String n) {
        monNom = n;
    }

    public void run() {
        System.out.println ("Bonjour de " + monNom);
        Thread.yield();
        System.out.println ("Au revoir de " + monNom);
    }
}

public class ExempleThread1 {
    public static void main(String []args) {
        uneTache t1, t2, t3;

        t1 = new uneTache("t1");
        t2 = new uneTache("t2");
        t3 = new uneTache("t3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```



```
$ java ExempleThread1
Bonjour de t1
Bonjour de t3
Au revoir de t3
Bonjour de t2
Au revoir de t1
Au revoir de t2
$ java ExempleThread1
Bonjour de t1
Bonjour de t3
Bonjour de t2
Au revoir de t3
Au revoir de t1
Au revoir de t2
```

Chronogramme pour l'exécution de la variation

6

main



```
public class ExempleThread1 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```



```
$ java ExempleThread1
```

Chronogramme pour l'exécution de la variation

6



```
public class ExempleThread1 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        ▶ t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays the command to run the Java program: `$ java ExempleThread1`.

Chronogramme pour l'exécution de la variation

6



```
public class ExempleThread1 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains the command: `$ java ExempleThread1`

Chronogramme pour l'exécution de la variation

6

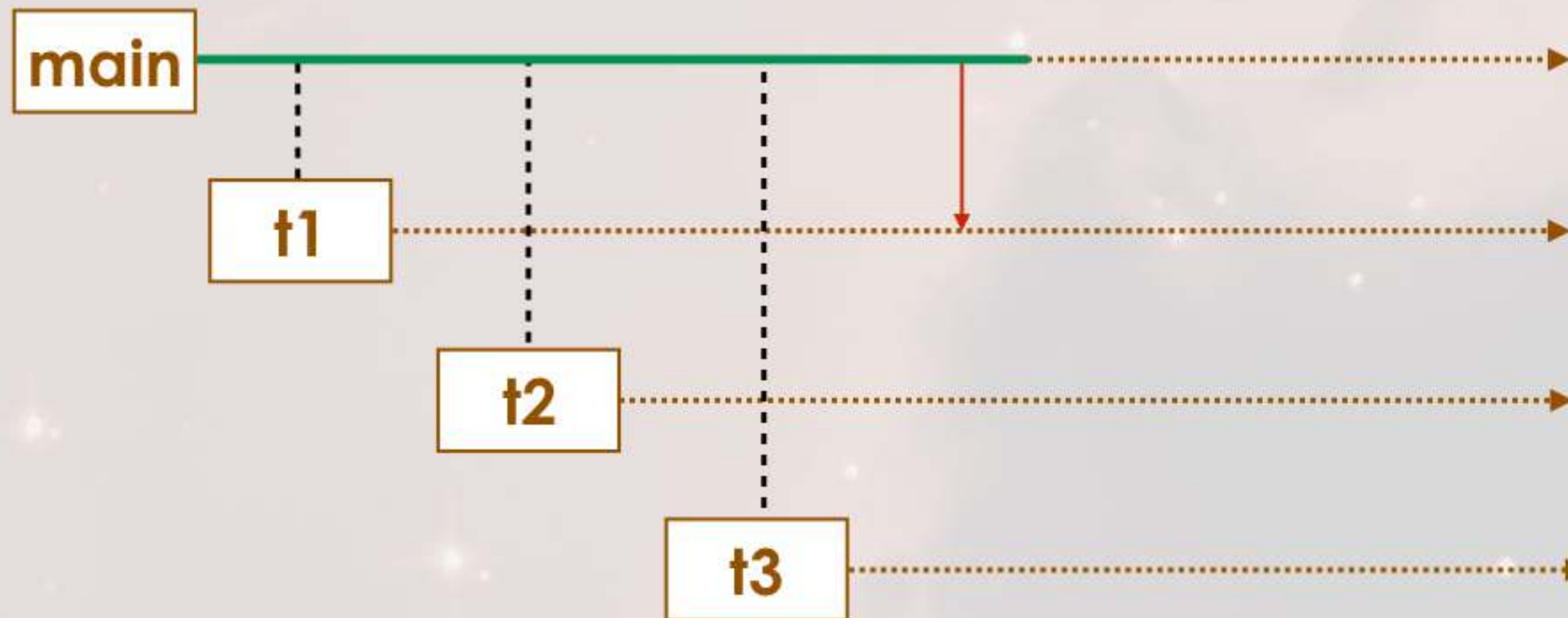


```
public class ExempleThread1 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleThread1
```

Chronogramme pour l'exécution de la variation

6

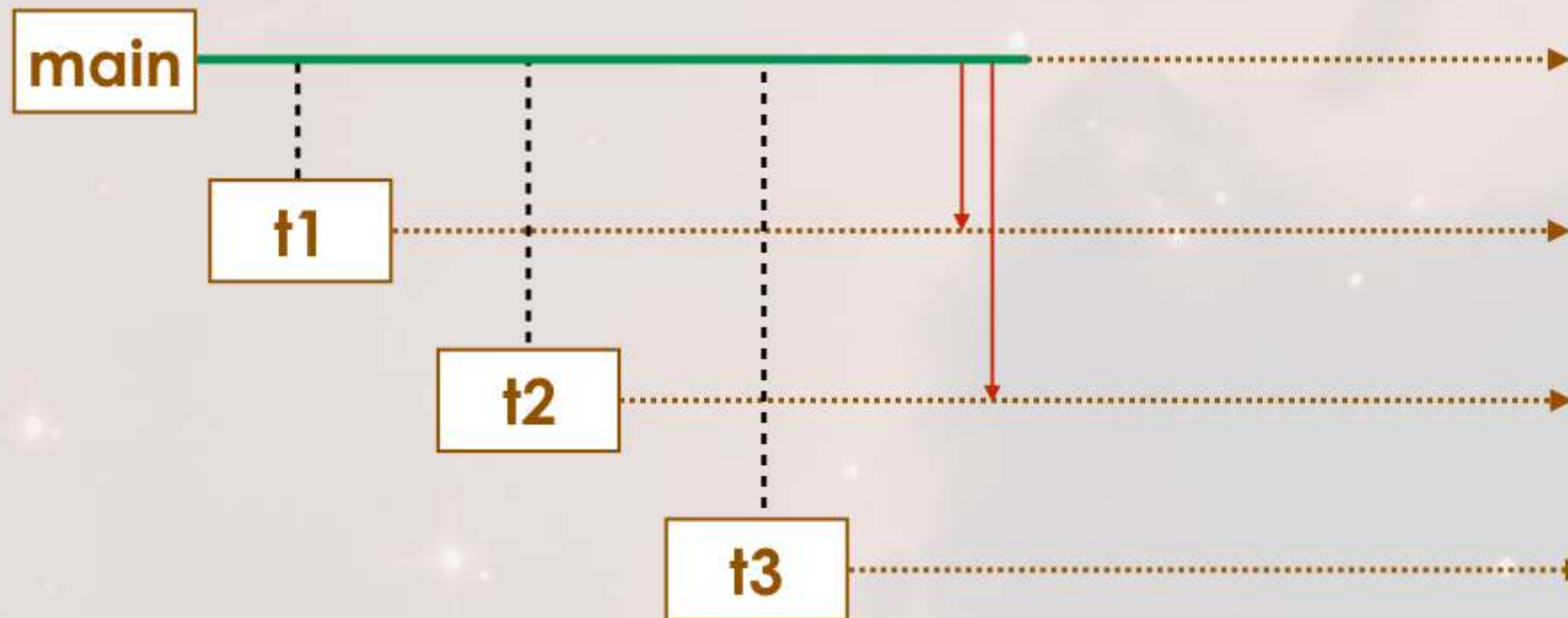


```
public class ExempleThread1 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleThread1
```

Chronogramme pour l'exécution de la variation

6



```
public class ExempleThread1 {  
    public static void main(String []args) {  
        uneTache t1, t2, t3;  
  
        t1 = new uneTache("t1");  
        t2 = new uneTache("t2");  
        t3 = new uneTache("t3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleThread1
```

Chronogramme pour l'exécution de la variation

6



```
public class ExempleThread1 {
    public static void main(String []args) {
        uneTache t1, t2, t3;

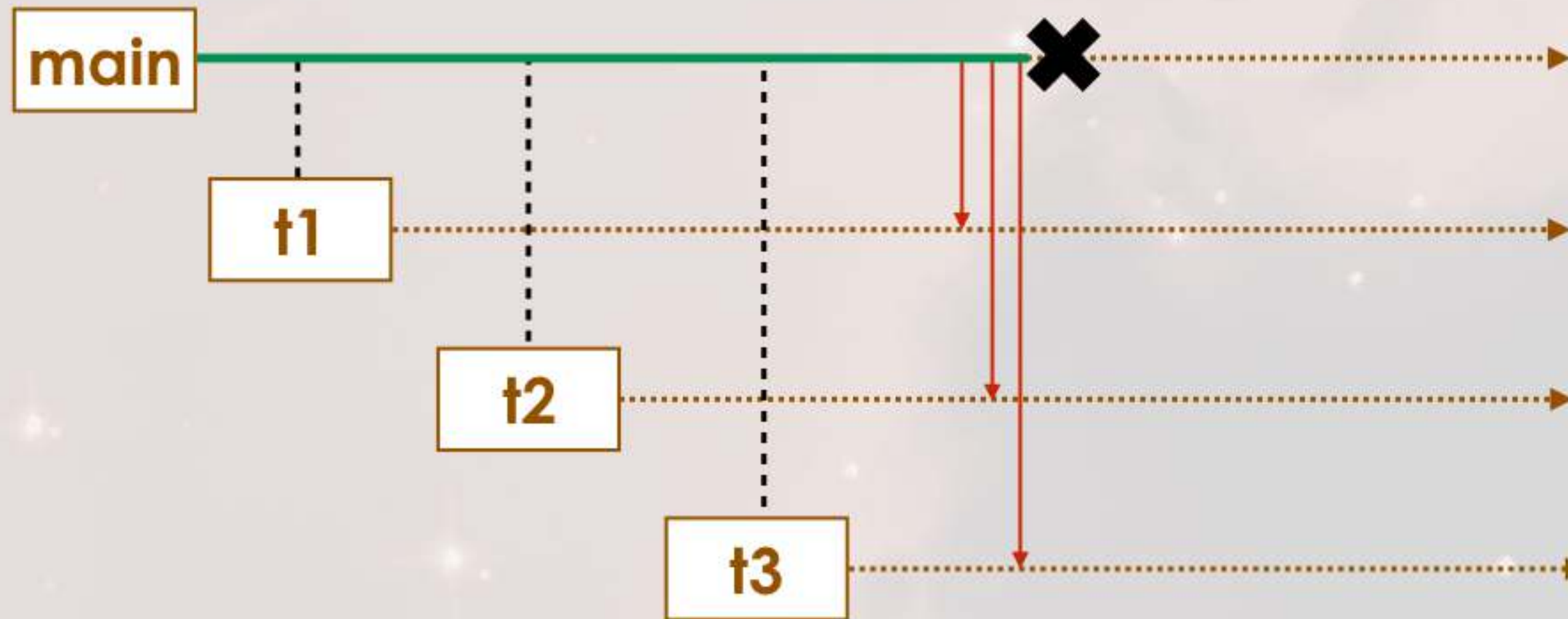
        t1 = new uneTache("t1");
        t2 = new uneTache("t2");
        t3 = new uneTache("t3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

```
$ java ExempleThread1
```

Chronogramme pour l'exécution de la variation

6

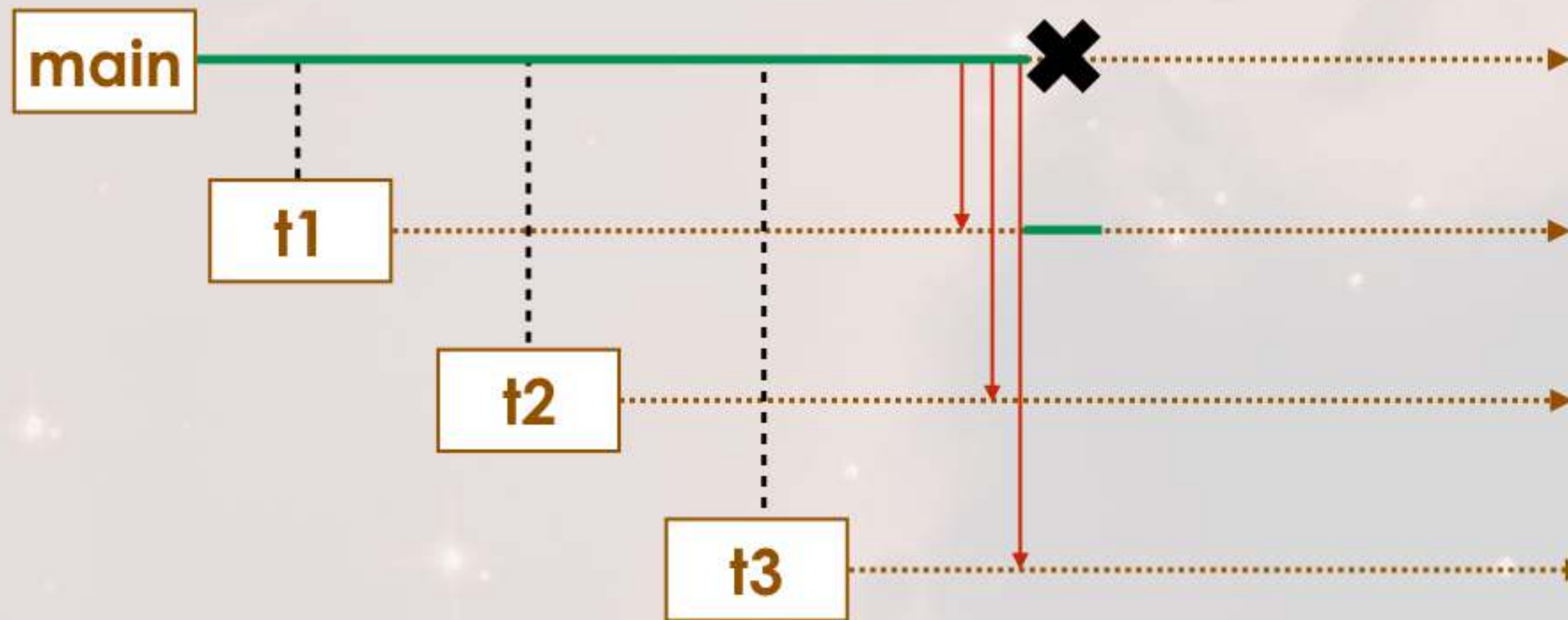


```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

```
$ java ExempleThread1
```

Chronogramme pour l'exécution de la variation

6

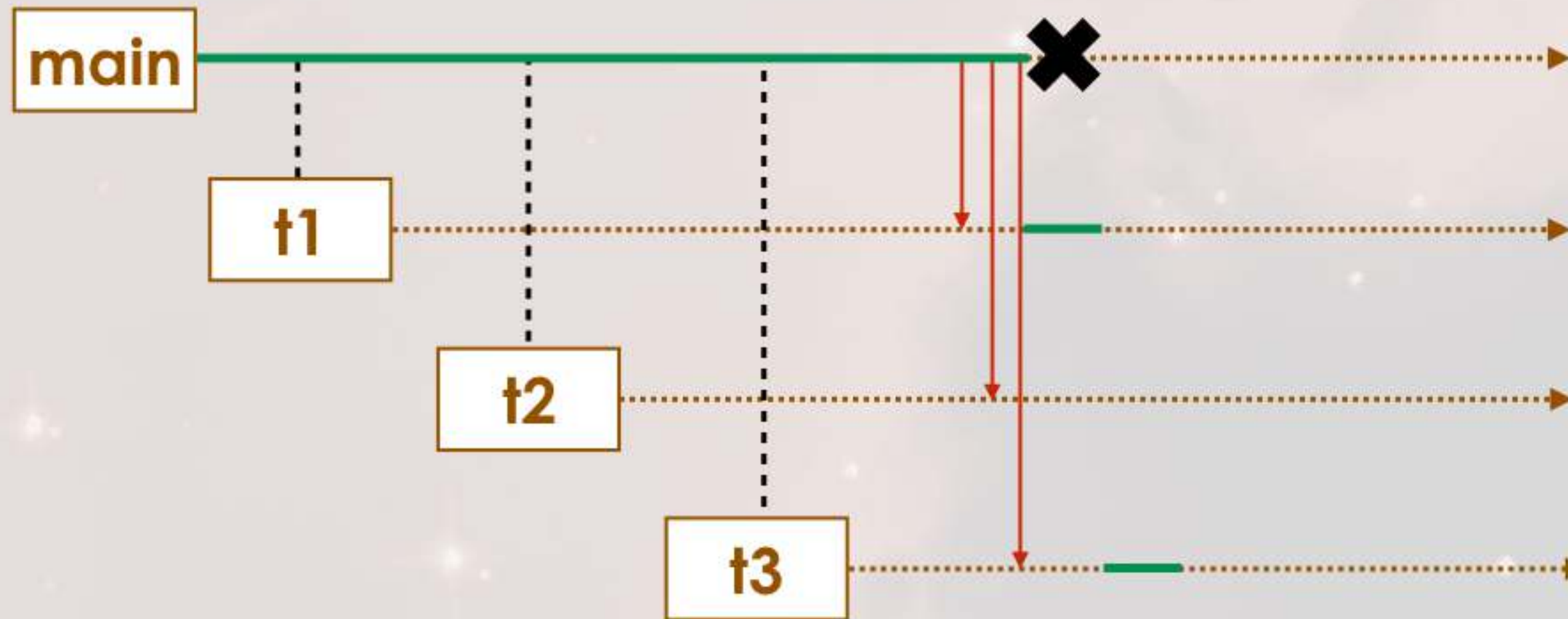


```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

```
$ java ExempleThread1  
Bonjour de t1
```

Chronogramme pour l'exécution de la variation

6

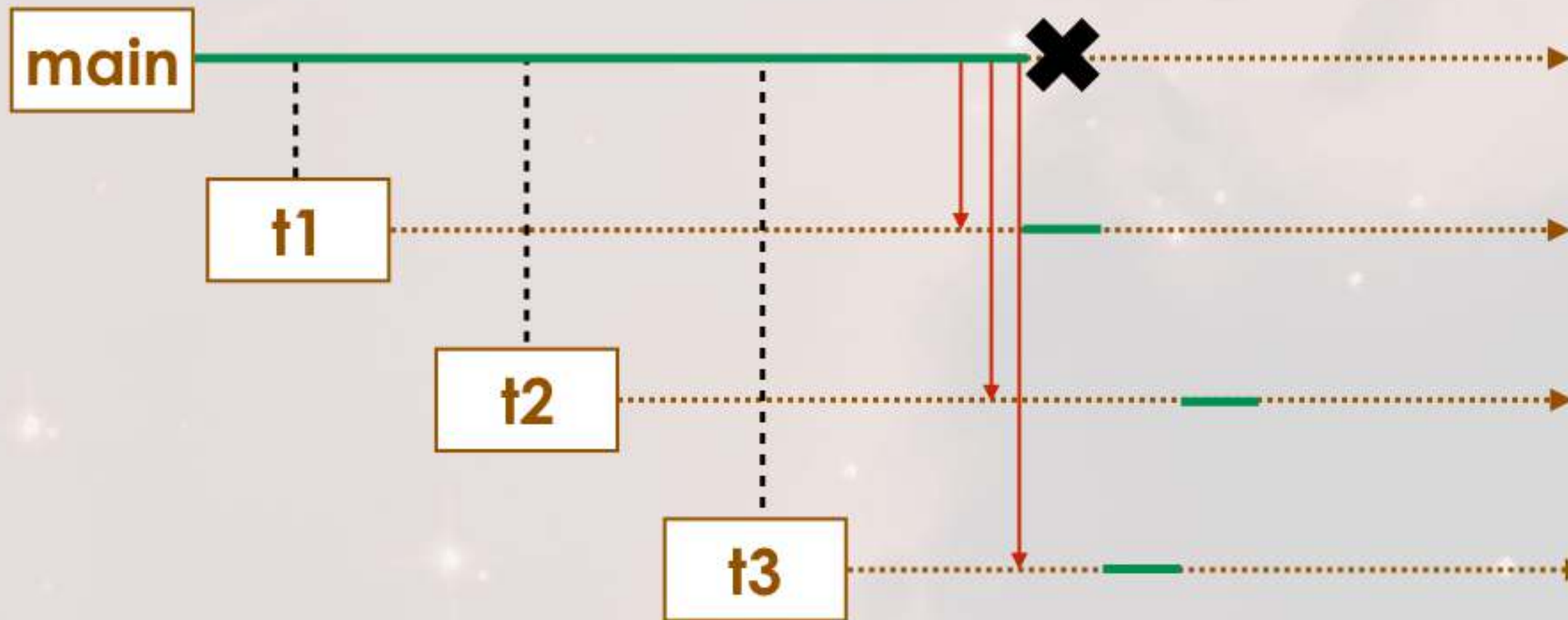


```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

```
$ java ExempleThread1  
Bonjour de t1  
Bonjour de t3
```

Chronogramme pour l'exécution de la variation

6

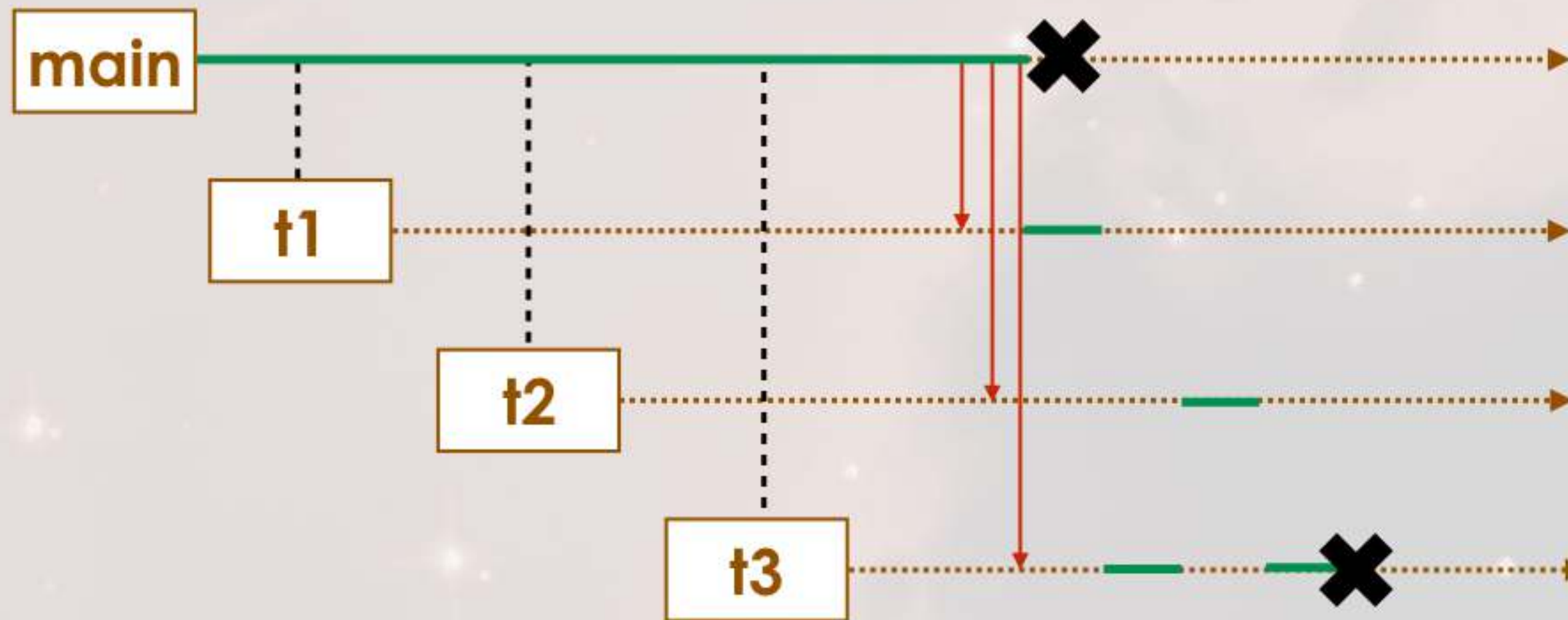


```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

```
$ java ExempleThread1  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2
```

Chronogramme pour l'exécution de la variation

6

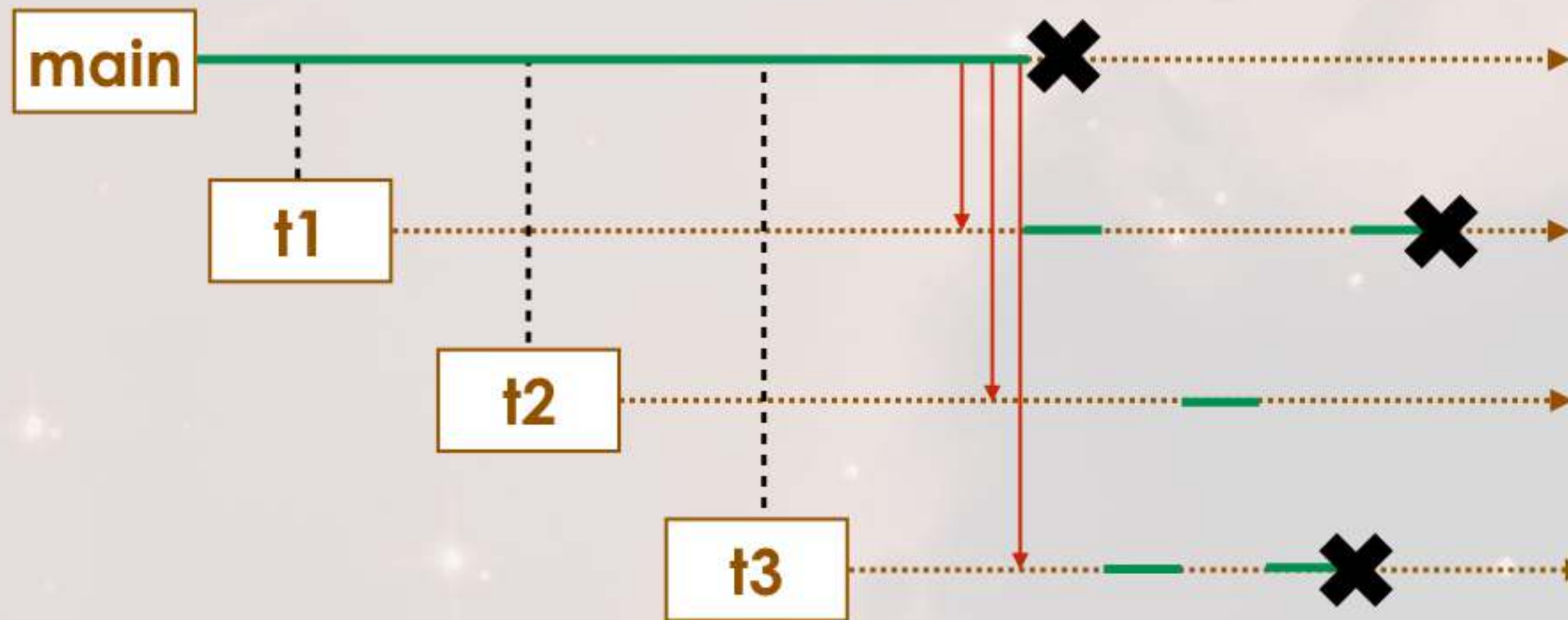


```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

```
$ java ExempleThread1  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2  
Au revoir de t3
```

Chronogramme pour l'exécution de la variation

6

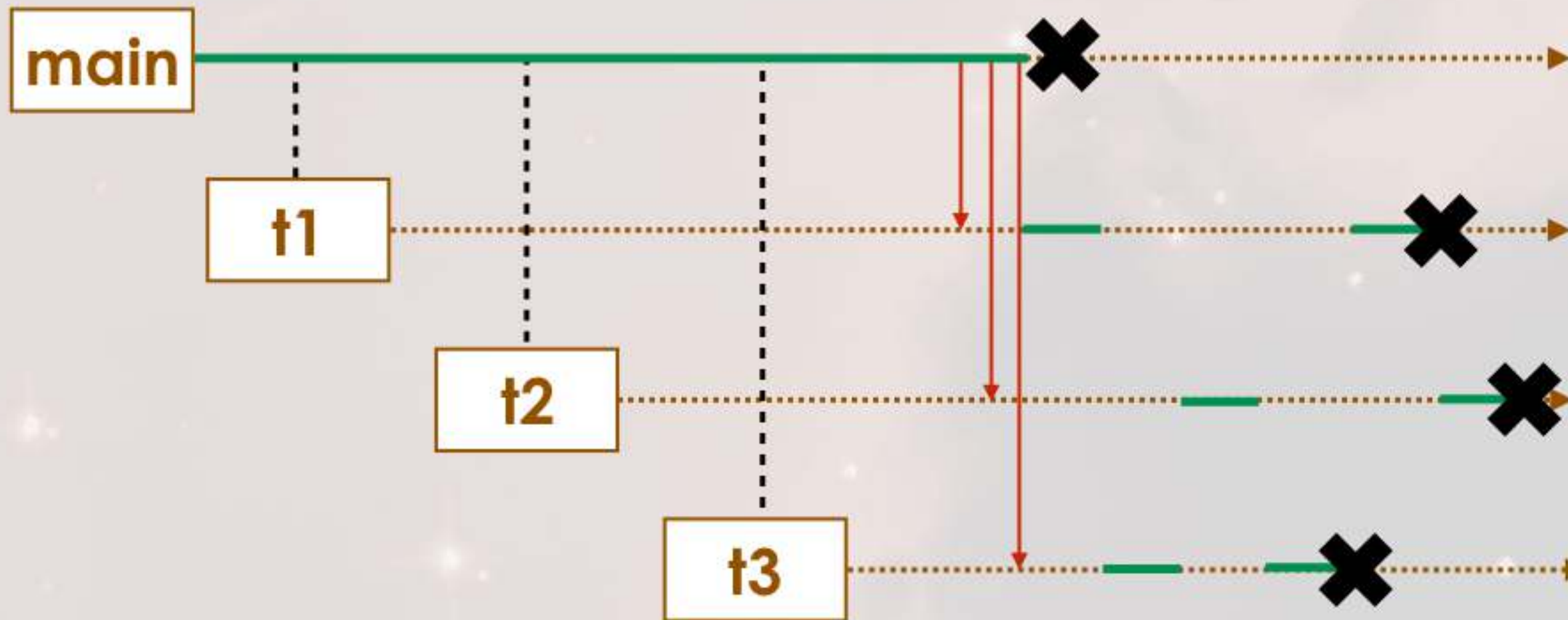


```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

```
$ java ExempleThread1  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2  
Au revoir de t3  
Au revoir de t1
```

Chronogramme pour l'exécution de la variation

6

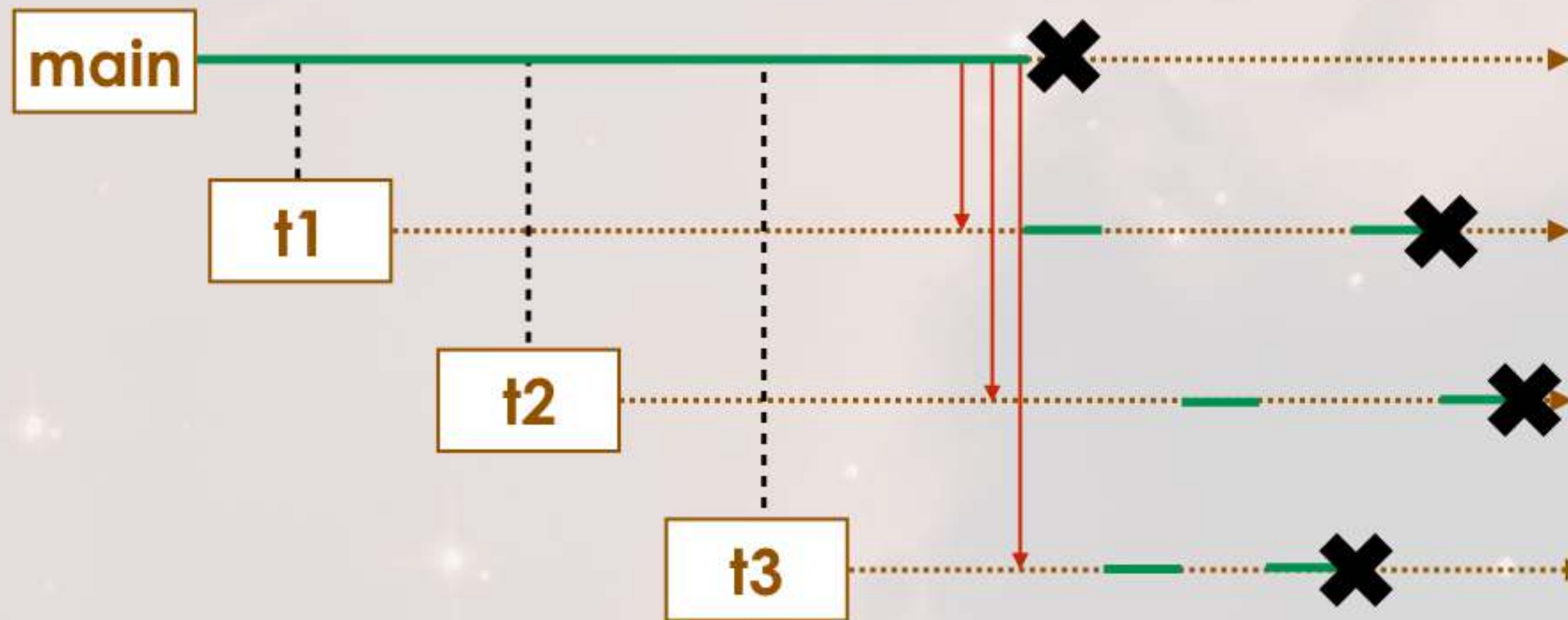


```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

```
$ java ExempleThread1  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2  
Au revoir de t3  
Au revoir de t1  
Au revoir de t2
```

Chronogramme pour l'exécution de la variation

6



```
public void run() {  
    System.out.println ("Bonjour de " + monNom);  
    Thread.yield();  
    System.out.println ("Au revoir de " + monNom);  
}
```

C'est fini!

Le programme se termine
Le processus est nettoyé par l'OS

```
$ java ExempleThread1  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2  
Au revoir de t3  
Au revoir de t1  
Au revoir de t2
```

En guise de conclusion...

7

Facile non?

-  Fil d'exécution = dérivation de Thread
-  Création d'un fil = instancier un objet

Communication?

-  On verra plus tard mais...

... appel de méthode

